NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR EMBEDDED SYSTEMS

LAB REPORT 6: VHDL CODE FOR MULTIPLEXER

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20th Nov	
2.	VHDL Code (OR & NOT Gate) using testbench	23rd Nov	
3.	Universal Gates and Special Gates	30th Nov	
4.	VHDL Code for Adder	7th Dec	
5.	VHDL Code for Subtractor	14th Dec	
6.	VHDL Code for Multiplexer	21st Dec	

LAB 6: VHDL CODE FOR MULTIPLEXER

OBJECTIVE:

The objective of this lab is to understand the working principle of a Multiplexer by designing and implementing it using VHDL. This experiment aims to familiarize students with combinational logic design, selection logic, and VHDL coding techniques for data routing circuits. It also helps in simulating and verifying the correct operation of the multiplexer for different input and select line combinations.

THEORY:

Introduction to Multiplexer

A Multiplexer (MUX) is a combinational digital circuit that selects one of several input signals and forwards it to a single output line based on the values of select lines. It is commonly used in digital systems for data selection, signal routing, resource sharing, and communication systems. Multiplexers reduce hardware complexity by allowing multiple signals to share a single output line.

Multiplexer

A multiplexer has:

- Multiple data inputs
- One output
- One or more select lines

The number of select lines determines the number of input lines. For an *n* select line multiplexer, the number of inputs is (2^n).

A **4:1 Multiplexer** is a combinational digital circuit that selects **one of four input lines** and routes it to a **single output line** based on the combination of **two select lines**. It is widely used in digital systems for data selection, signal routing, and logic implementation.

Inputs

- Data inputs: I0, I1, I2, I3
- Select lines: S1, S0

Output

- Y

Working Principle

The output of a 4:1 multiplexer depends on the binary value of the select lines. For each unique combination of S1 and S0, only one input is connected to the output.

Logic Equation

$$Y = I0 \cdot \overline{S1} \cdot \overline{S0} + I1 \cdot \overline{S1} \cdot S0 + I2 \cdot S1 \cdot \overline{S0} + I3 \cdot S1 \cdot S0$$

Truth Table

S1	S0	Output (Y)
0	0	I0
0	1	I1
1	0	I2
1	1	I3

A 4:1 multiplexer can be implemented using logic gates or modeled efficiently using VHDL with conditional assignments or case statements.

8:1 MULTIPLEXER

An **8:1 Multiplexer** is a combinational digital circuit that selects **one of eight input lines** and connects it to a **single output line** using **three select lines**. It is commonly used in complex digital systems where multiple data sources need to be routed through a single channel.

Inputs

- Data inputs: I0, I1, I2, I3, I4, I5, I6, I7
- Select lines: S2, S1, S0

Output

- Y

Working Principle

The output is determined by the binary value of the three select lines. Each combination selects exactly one input among the eight available inputs.

Logic Equation

$$\begin{aligned} Y = & I0 \cdot \overline{S2} \cdot \overline{S1} \cdot \overline{S0} + I1 \cdot \overline{S2} \cdot \overline{S1} \cdot S0 + \\ & I2 \cdot \overline{S2} \cdot S1 \cdot \overline{S0} + I3 \cdot \overline{S2} \cdot S1 \cdot S0 + \\ & I4 \cdot S2 \cdot \overline{S1} \cdot \overline{S0} + I5 \cdot S2 \cdot \overline{S1} \cdot S0 + \\ & I6 \cdot S2 \cdot S1 \cdot \overline{S0} + I7 \cdot S2 \cdot S1 \cdot S0 \end{aligned}$$

Truth Table

S2	S1	S0	Output (Y)
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3

1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

An 8:1 multiplexer can be constructed using smaller multiplexers or implemented directly using VHDL, making it suitable for large data routing applications.

Multiplexer Implementation Using VHDL

VHDL (VHSIC Hardware Description Language) is used to describe the behavior and structure of digital circuits. In this lab, VHDL is used to model a multiplexer using conditional signal assignments or case statements. The select lines control which input is routed to the output. Simulation of the VHDL code helps in verifying the correct functionality of the multiplexer for all possible combinations of inputs and select lines.

DATAFLOW FOR 4:1 MUX:

```
library IEEE;
use IEEE.std_logic_1164.all;
entity arpmuxd is
  port(
  I : in std_logic_vector(3 downto 0);
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic
  );
end arpmuxd;
architecture behavioral of arpmuxd is
begin
```

```
  Y <= (not S(1) and not S(0) and I(0))
  or (not S(1) and S(0) and I(1))
  or (S(1) and not S(0) and I(2))
  or (S(1) and S(0) and I(3));
end behavioral;
```

BEHAVIORAL FOR 4:1 MUX

```
library IEEE;
use IEEE.std_logic_1164.all;
entity arpmux is
  port(
  I : in std_logic_vector(3 downto 0);
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic
  );
end arpmux;
architecture behavioral of arpmux is
begin
```

```
  process(I,S)
    begin
    case S is
      when "00" => Y<=I(0);
      when "01" => Y<=I(1);
      when "10" => Y<=I(2);
      when "11" => Y<=I(3);
      when others => Y<='0';
    end case;
  end process;
end behavioral;
```

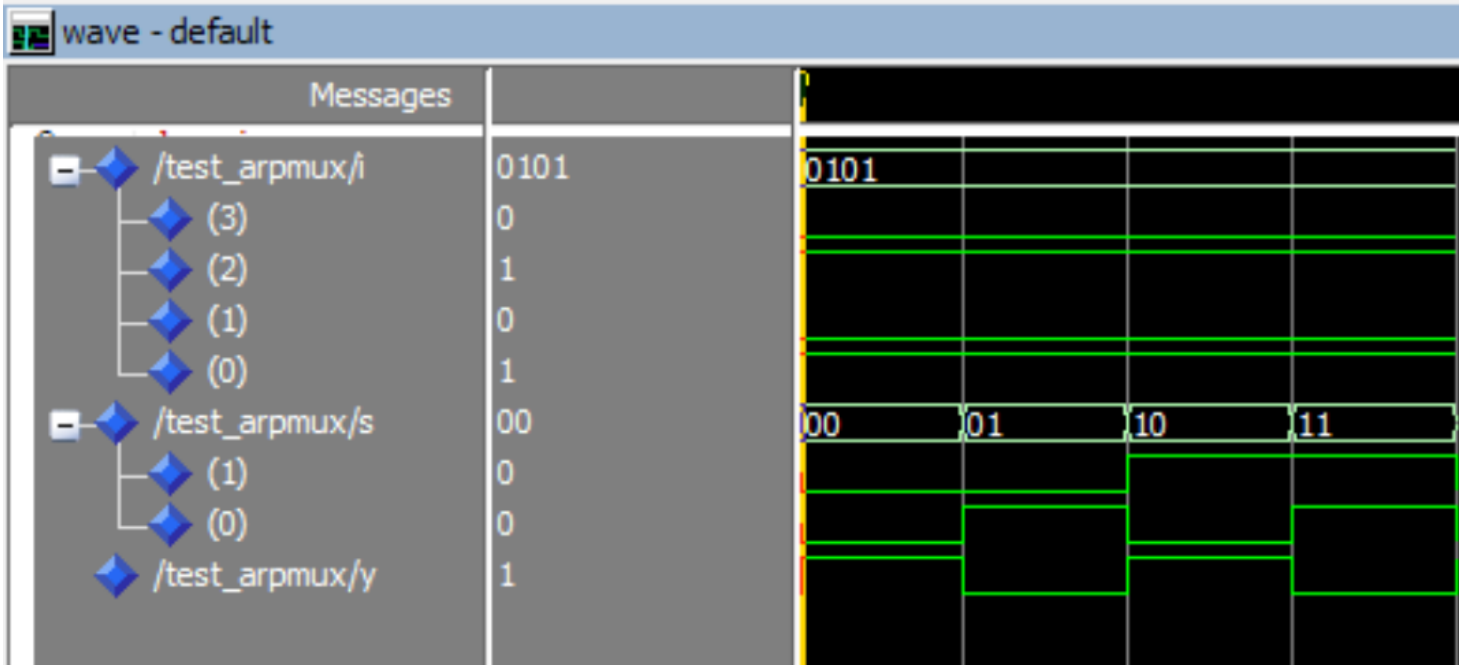
TESTBENCH FOR 4:1 MUX

```
library IEEE;
use IEEE.std_logic_1164.all;
entity test_arpmux is
end test_arpmux;
architecture arch_mux of test_arpmux is
component arpmux is
  port(
  I : in std_logic_vector(3 downto 0);
  S : in std_logic_vector(1 downto 0);
  Y : out std_logic
```

```
  );
end component;
signal I : std_logic_vector(3 downto 0) := "0101";
signal S : std_logic_vector(1 downto 0);
signal Y : std_logic;
begin
  UUT: arpmux port map (
    I => I,
    S => S,
    Y => Y
  );

  stim_proc : process
  begin
    S <= "00";
    wait for 100 ps;
    S <= "01";
    wait for 100 ps;
    S <= "10";
    wait for 100 ps;
    S <= "11";
    wait for 100 ps;
  end process;
end arch_mux;
```

WAVEFORM FOR 4:1 MUX



DATAFLOW FOR 8:1 MUX

```
library IEEE;
use IEEE.std_logic_1164.all;

entity arpmuxd is
  port(
    I : in std_logic_vector(7 downto 0);
    S : in std_logic_vector(2 downto 0);
    Y : out std_logic
  );
end arpmuxd;

architecture behavioral of arpmuxd is
begin
  Y <= (not S(2) and not S(1) and not S(0) and I(0)) or
       (not S(2) and not S(1) and     S(0) and I(1)) or
       (not S(2) and     S(1) and not S(0) and I(2)) or
       (not S(2) and     S(1) and     S(0) and I(3)) or
       (    S(2) and not S(1) and not S(0) and I(4)) or
       (    S(2) and not S(1) and     S(0) and I(5)) or
       (    S(2) and     S(1) and not S(0) and I(6)) or
       (    S(2) and     S(1) and     S(0) and I(7));
end behavioral;
```


```
library IEEE;
use IEEE.std_logic_1164.all;
entity arpmuxd is
  port(
    I : in std_logic_vector(7 downto 0);
    S : in std_logic_vector(2 downto 0);
    Y : out std_logic
  );
end arpmuxd;
architecture behavioral of arpmuxd is
begin
  Y <= (not S(2) and not S(1) and not S(0) and I(0)) or
       (not S(2) and not S(1) and     S(0) and I(1)) or
       (not S(2) and     S(1) and not S(0) and I(2)) or
       (not S(2) and     S(1) and     S(0) and I(3)) or
       (    S(2) and not S(1) and not S(0) and I(4)) or
       (    S(2) and not S(1) and     S(0) and I(5)) or
       (    S(2) and     S(1) and not S(0) and I(6)) or
       (    S(2) and     S(1) and     S(0) and I(7));
end behavioral;
```


BEHAVIORAL FOR 8:1 MUX

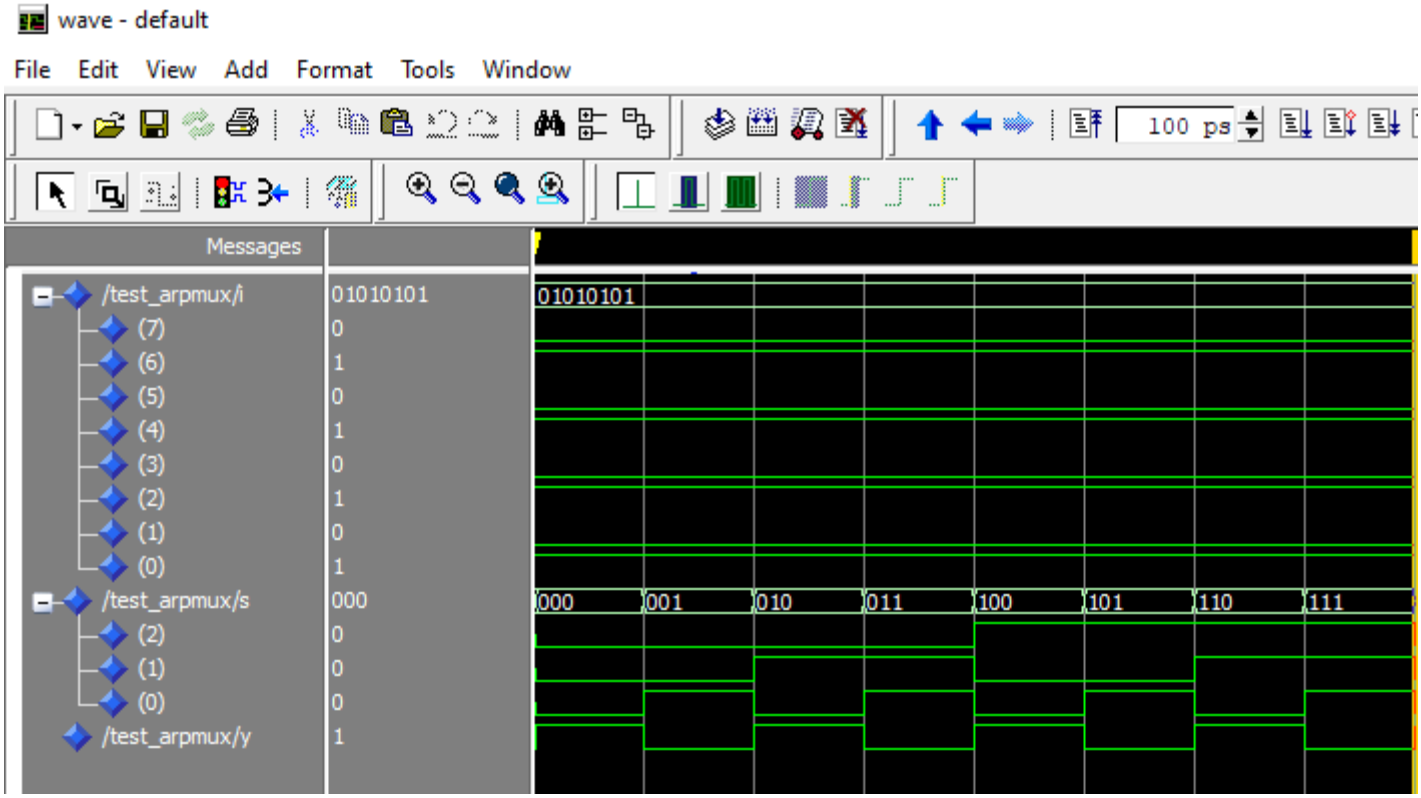
```
library IEEE;
use IEEE.std_logic_1164.all;
entity arpmux is
  port(
  I : in std_logic_vector(7 downto 0);
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic
  );
end arpmux;
architecture behavioral of arpmux is
begin
  process(I,S)
    begin
    case S is
      when "000" => Y<=I(0);
      when "001" => Y<=I(1);
      when "010" => Y<=I(2);
      when "011" => Y<=I(3);
      when "100" => Y<=I(4);
      when "101" => Y<=I(5);
      when "110" => Y<=I(6);
      when "111" => Y<=I(7);
      when others => Y<='0';
    end case;
  end process;
end behavioral;
```

TESTBENCH FOR 8:1 MUX

```
library IEEE;
use IEEE.std_logic_1164.all;
entity test_arpmux is
end test_arpmux;
architecture arch_mux of test_arpmux is
component arpmux is
  port(
  I : in std_logic_vector(7 downto 0);
  S : in std_logic_vector(2 downto 0);
  Y : out std_logic
  );
end component;
signal I : std_logic_vector(7 downto 0) := "01010101";
signal S : std_logic_vector(2 downto 0);
signal Y : std_logic;
begin
  UUT: arpmux port map (
    I => I,
    S => S,
    Y => Y
  );

  stim_proc : process
  begin
    S <= "000"; wait for 100 ps;
    S <= "001"; wait for 100 ps;
    S <= "010"; wait for 100 ps;
    S <= "011"; wait for 100 ps;
     S <= "100"; wait for 100 ps;
    S <= "101"; wait for 100 ps;
    S <= "110"; wait for 100 ps;
    S <= "111"; wait for 100 ps;
  end process;
end arch_mux;
```

WAVEFORM FOR 8:1 MUX


wave - default
File Edit View Add Format Tools Window
100 ps
Messages
/test_arpmux/i
01010101
(7)
0
(6)
1
(5)
0
(4)
1
(3)
0
(2)
1
(1)
0
(0)
1
/test_arpmux/s
000
(2)
0
(1)
0
(0)
0
/test_arpmux/y
1
01010101
000
001
010
011
100
101
110
111

RESULT:

The 4:1 and 8:1 Multiplexer circuits were successfully designed and implemented using VHDL. The simulation was performed for all possible combinations of select lines and input signals. The observed output correctly followed the selected input line for each select line combination. The simulation waveforms confirmed that only one input was routed to the output at a time, as per the theoretical truth tables of the multiplexers.

CONCLUSION:

In this experiment, the working of 4:1 and 8:1 Multiplexers was successfully studied and verified using VHDL. The experiment helped in understanding the concept of data selection using select lines and the role of multiplexers in digital systems. The results obtained from simulation matched the expected theoretical behavior, proving the correctness of the VHDL implementation. This experiment provides a strong foundation for designing complex data routing and combinational logic circuits using multiplexers.